CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G5
SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: EventWear Lease System

Prepared By: Jasmine Ciely N. Palicte

Date of Submission:

Version:

Table of Contents

1. Introduction 3
   1.1. Purpose 3
   1.2. Scope 3
   1.3. Definitions, Acronyms, and Abbreviations 3
2. Overall Description 4
   2.1. System Perspective 4
   2.2. User Classes and Characteristics 4
   2.3. Operating Environment 5
   2.4. Assumptions and Dependencies 6
3. System Features and Functional Requirements 7
   3.1. Feature 1: 7
   3.2. Feature 2: 9
4. Non-Functional Requirements 12
5. System Models (Diagrams) 14
   5.1. ERD 14
   5.2. Use Case Diagram 14
   5.3. Activity Diagram 15
   5.4. Class Diagram 16
   5.5. Sequence Diagram 17
6. Appendices 18

1. Introduction

1.1. Purpose

This document provides a comprehensive overview of the **Event Wear Rental System**, a full-stack web application for renting premium event outfits. It describes the system's architecture, features, user interactions, and technical requirements. This document is intended for developers, project managers, stakeholders, and quality assurance teams to ensure a unified understanding of the system.

1.2. Scope

The Event Wear Rental System is a complete web application consisting of:

- Frontend: React-based single-page application (SPA) with responsive design
- Backend: Spring Boot REST API with session-based authentication
- Database: Relational database for persistent storage

The system supports three user modes:

1. **Guest Users**: Browse outfits, view details, check availability (read-only)

2. **Authenticated Users**: Full access including bookings, profile management, and notifications

3. **Administrators**: System management capabilities (endpoints exist, UI pending)

The system handles user authentication, session management, outfit catalog browsing, and provides a foundation for booking functionality.

1.3. Definitions, Acronyms, and Abbreviations

Term	Definition
SPA	Single Page Application - Frontend architecture where content loads dynamically without full page reloads
REST API	Representational State Transfer Application Programming Interface
JWT	JSON Web Token (not used - system uses server-side sessions)
CORS	Cross-Origin Resource Sharing - Security mechanism for cross-domain requests
Session	Server-side storage of user state between HTTP requests
Guest User	Unauthenticated user with limited, read-only access
Authenticated User	Registered user with full access privileges
Outfit	Rentable clothing item (gown, barong, tuxedo, etc.)
Booking	Rental reservation for a specific outfit and date range

2. Overall Description

2.1. System Perspective

The Event Wear Rental System is a standalone full-stack application with:

- **Frontend (React)**:
  - Runs on http://localhost:5173 (development)
  - Manages client-side state for authentication
  - Implements guest mode without backend authentication
  - Communicates with backend via Axios with credentials enabled
- **Backend (Spring Boot)**:
  - Runs on http://localhost:8080
  - Provides REST API endpoints
  - Manages user sessions with Spring Security
  - Handles authentication via session cookies
- **Database (JPA/Hibernate)**:
  - Stores user accounts and outfit inventory
  - Manages relationships between entities

2.2. User Classes and Characteristics

1. Guest User

- **Characteristics**: Not logged in, exploring the platform
- **Capabilities**:
  - View outfit catalog (all outfits)
  - Filter by category
  - View outfit details and pricing
  - Check availability (read-only)
  - See "sample" data for bookings and stats
- **Limitations**: Cannot create bookings, cannot access restricted tabs (My Bookings, Notifications, Profile)

- **Promotion**: Visual prompts to create account throughout the interface

2. Authenticated User

- **Characteristics**: Registered and logged in
- **Capabilities**:
    - All guest capabilities
    - View personal dashboard with real stats
    - Manage profile information
    - View real booking history
    - Receive notifications
    - Book outfits (endpoint ready, UI pending)
- **Session**: Server-managed session with one session per user limit

3. Administrator

- **Characteristics**: Users with ROLE_ADMIN authority
- **Capabilities**:
    - All authenticated user capabilities
    - Access to /api/admin/** endpoints
    - Manage outfit inventory (add, edit, delete)
    - Manage all bookings
    - User management (pending)

2.3. Operating Environment

Frontend Requirements:

- **Runtime**: Node.js 16+ with npm/yarn
- **Browser**: Modern browsers (Chrome, Firefox, Safari, Edge) with JavaScript enabled
- **Dependencies**: React 18+, React Router 6+, Axios, Lucide React (icons)
- **Development Server**: Vite (default port 5173)

Backend Requirements:

- **Runtime**: Java 17+ (JDK 17 or later)
- **Framework**: Spring Boot 3.x
- **Web Server**: Embedded Tomcat (port 8080)
- **Database**: MySQL, PostgreSQL, or H2 (development)
- **Build Tool**: Maven

Network Requirements:

- CORS configured for http://localhost:3000, http://localhost:5173
- Credentials enabled (withCredentials: true in Axios, allowCredentials: true in backend)
- Session cookies enabled in browser

2.4. Assumptions and Dependencies

Assumptions:

1. Users have valid email addresses for registration
2. Backend server is running before frontend attempts API calls
3. Users understand basic navigation of web applications
4. Guest mode is a client-side concept only (no server-side guest sessions)
5. The frontend and backend are deployed on separate ports during development

Dependencies:

1. **Backend depends on**:
    - Database availability and connectivity
    - Proper CORS configuration for frontend origin
    - Spring Security configuration for session management
2. **Frontend depends on**:
    - Backend API availability at http://localhost:8080
    - LocalStorage for persisting guest state
    - Browser cookies for session management
3. **Critical Configuration**:

    - CORS must allow frontend origin with credentials
    - Session cookies must have proper SameSite and Secure attributes in production
    - CSRF protection must be enabled in production

3. System Features and Functional Requirements

Describe each major feature of the system and its functional requirements.

3.1. Feature 1: User Authentication & Session Management

Description: Complete authentication system supporting registration, login, logout, and session management. Guest mode provides limited access without authentication.

Functional Requirements:

- **FR 1.1**: User Registration
    - /api/auth/register endpoint accepts firstName, lastName, email, password, confirmPassword
    - Validates email uniqueness and password match
    - Passwords stored using BCrypt encryption
    - Returns success message upon completion
- **FR 1.2**: User Login
    - /api/auth/login endpoint validates credentials
    - Creates HTTP session on successful authentication
    - Stores user details in session for future requests
    - Returns UserResponse object (id, name, email, role)
    - Limits to one active session per user
- **FR 1.3**: Authentication Check
    - /api/auth/check endpoint verifies current session
    - Returns user details if authenticated
    - Used by frontend to restore session on page refresh
- **FR 1.4**: Logout
    - /api/auth/logout endpoint invalidates session

    - Clears Spring Security context
    - Frontend clears localStorage on logout
- **FR 1.5**: Guest Mode
    - Frontend-only guest mode without server authentication
    - Guest users can browse but cannot access restricted features
    - Guest state persisted in localStorage with role "GUEST"
    - Visual prompts encourage account creation
- **FR 1.6**: Session Persistence
    - Frontend checks authentication on app mount
    - Uses /api/auth/check to validate existing sessions
    - Falls back to localStorage if server unavailable
    - Redirects based on authentication status

3.2. Feature 2: Outfit Catalog Browsing

Description: Public catalog allowing all users (including guests) to browse available outfits with filtering and search capabilities.

Functional Requirements:

- FR 2.1: View All Outfits
    - GET /api/outfits returns complete outfit list
    - Frontend displays in responsive grid layout
    - Outfits include name, category, description, price, image, sizes
- FR 2.2: Popular Outfits
    - GET /api/outfits/popular returns featured outfits
    - Displayed prominently on dashboard
    - Marked with "Popular" badge in UI
- FR 2.3: Category Filtering
    - GET /api/outfits/category/{category} returns filtered results
    - Categories: Wedding, Barong, Gown, Cocktail, Formal, Casual
    - Frontend category navigation implemented
- FR 2.4: Outfit Details
    - GET /api/outfits/{id} returns single outfit details
    - Displays all attributes, sizes, and availability
    - Quick view modal for fast preview
- FR 2.5: Availability Check
    - GET /api/outfits/{id}/availability checks date range availability
    - Returns availability status and pricing
    - Guest users receive prompt to sign in for booking
- FR 2.6: Show More/Less Functionality
    - Initial display shows 8 outfits (2 rows of 4)

    - "Show More" button reveals all outfits
    - "Show Less" returns to limited view

3.3. Feature 3: Dashboard & User Experience

Description: Personalized dashboard displaying relevant information based on user type (guest or authenticated).

Functional Requirements:

- **FR 3.1**: User Greeting
    - Displays personalized welcome message
    - Shows "Welcome back, [Name]!" for authenticated users
    - Shows "Welcome, Guest!" for guest users
- **FR 3.2**: Statistics Display
    - **Guest Stats**: Shows available outfits count (500+), categories (8), sign-up bonus (100 points)
    - **Authenticated Stats**: Shows active bookings count, total rentals, loyalty points with progress bar
    - Visual icons and trend indicators
- **FR 3.3**: Tab Navigation
    - Overview (accessible to all)
    - My Bookings (restricted, requires authentication)
    - Notifications (restricted)
    - Profile (restricted)
    - Lock icons displayed for restricted tabs to guests
- **FR 3.4**: Guest Restriction Handling
    - Restricted tabs show upgrade prompts
    - "Create Account" buttons trigger navigation to auth page with signup tab
    - Visual overlays indicate premium features
- **FR 3.5**: Recent Bookings Display
    - **Guest**: Shows sample bookings with lock overlay

    - **Authenticated**: Shows real booking history with status badges
    - Clicking locked items triggers upgrade prompt
- **FR 3.6**: Profile Management
    - Authenticated users can view their profile information
    - Displays name, email, role, member since date
    - Guest users see restricted message with upgrade option

3.4. Feature 4: Guess-to-Authenticated Conversion

Description: Strategic prompts throughout the application encouraging guest users to create accounts.

Functional Requirements:

- **FR 4.1**: Guest Banner
    - Prominent banner on dashboard for guest users
    - Message: "You're Browsing as a Guest"
    - "Create Account" button prominently displayed
- **FR 4.2**: Call-to-Action Buttons
    - "Sign in to Book" on outfit cards for guests
    - "Create Account" in stats grid for guests
    - "Sign In" button in header for guests
- **FR 4.3**: Navigation Preservation
    - When upgrading from guest to authenticated, preserve browsing context
    - Navigate to auth page with signup tab pre-selected
    - Clear guest localStorage before redirect
- **FR 4.4**: Seamless Transition
    - After authentication, redirect back to dashboard
    - Preserve intended action (e.g., booking specific outfit)

3.5. Feature 5: Profile & User Management

Description: User profile management for authenticated users.

Functional Requirements:

- **FR 5.1**: View Profile
  - GET /api/user/me returns current user details
  - Frontend displays in Profile tab
  - Shows name, email, role, join date
- **FR 5.2**: Edit Profile (Future)
  - Update personal information
  - Change password functionality
  - Manage preferences

4. Non-Functional Requirements

Security

- **NFR 1.1**: Passwords must be encrypted using BCrypt (10+ rounds)
- **NFR 1.2**: Session cookies must have HttpOnly, Secure, and SameSite flags in production
- **NFR 1.3**: CSRF protection must be enabled in production (currently disabled for development)
- **NFR 1.4**: CORS must only allow trusted origins
- **NFR 1.5**: Input validation on both frontend and backend
- **NFR 1.6**: SQL injection prevention through JPA/Hibernate
- **NFR 1.7**: XSS protection through React's automatic escaping

Performance

- **NFR 2.1**: API response times under 500ms for standard operations
- **NFR 2.2**: Frontend initial load under 3 seconds on broadband
- **NFR 2.3**: Image lazy loading for outfit catalog
- **NFR 2.4**: Support for 100+ concurrent users

Usability

- **NFR 3.1**: Responsive design for mobile, tablet, and desktop

- **NFR 3.2**: Intuitive navigation with visual feedback on actions
- **NFR 3.3**: Loading states and spinners for async operations
- **NFR 3.4**: Clear error messages for validation failures
- **NFR 3.5**: Form validation with real-time feedback

Reliability

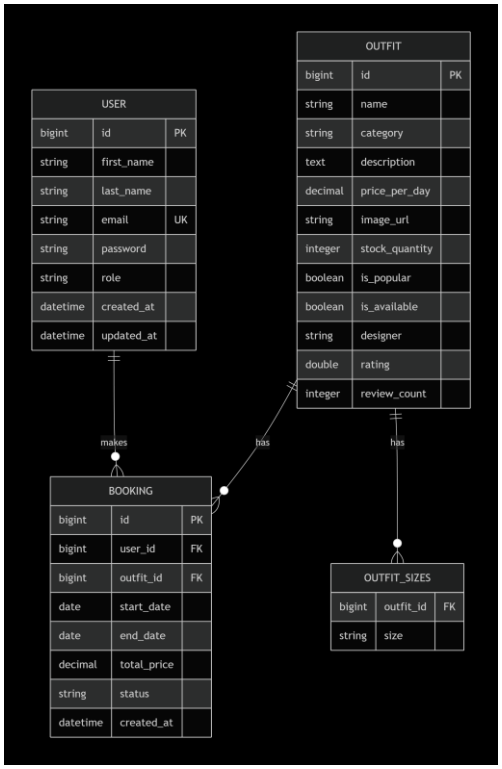
- **NFR 4.1**: 99.9% uptime for core functionality
- **NFR 4.2**: Graceful degradation when backend is unavailable
- **NFR 4.3**: Session persistence across page refreshes
- **NFR 4.4**: Data consistency with database transactions

Maintainability

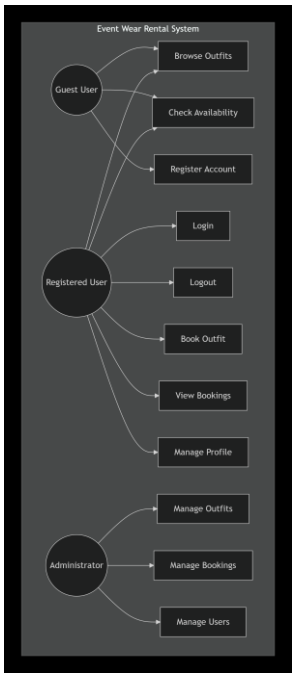
- **NFR 5.1**: Modular component structure in frontend
- **NFR 5.2**: Clear separation of concerns in backend (Controller-Service-Repository)
- **NFR 5.3**: Comprehensive error logging
- **NFR 5.4**: Environment-based configuration

5. System Models (Diagrams)

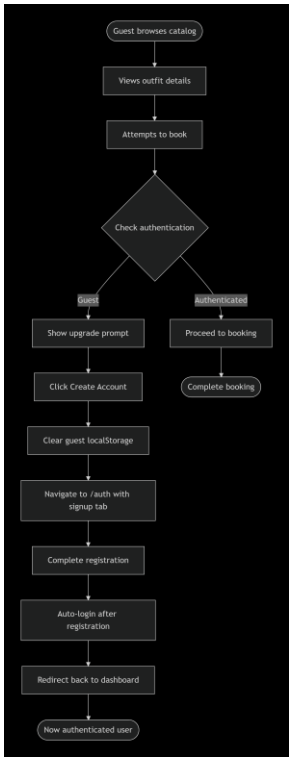
5.1. ERD



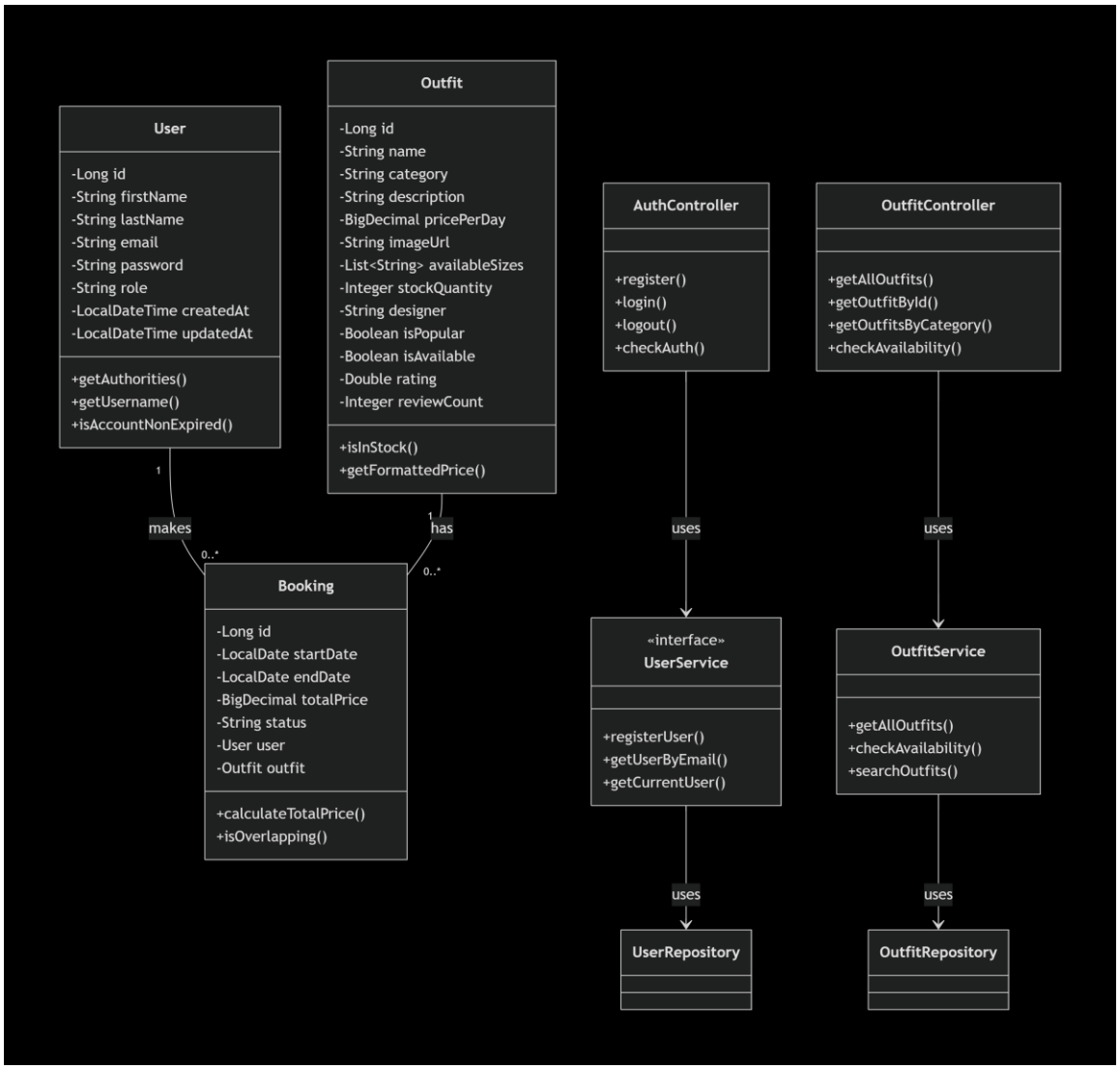
5.2. Use Case Diagram



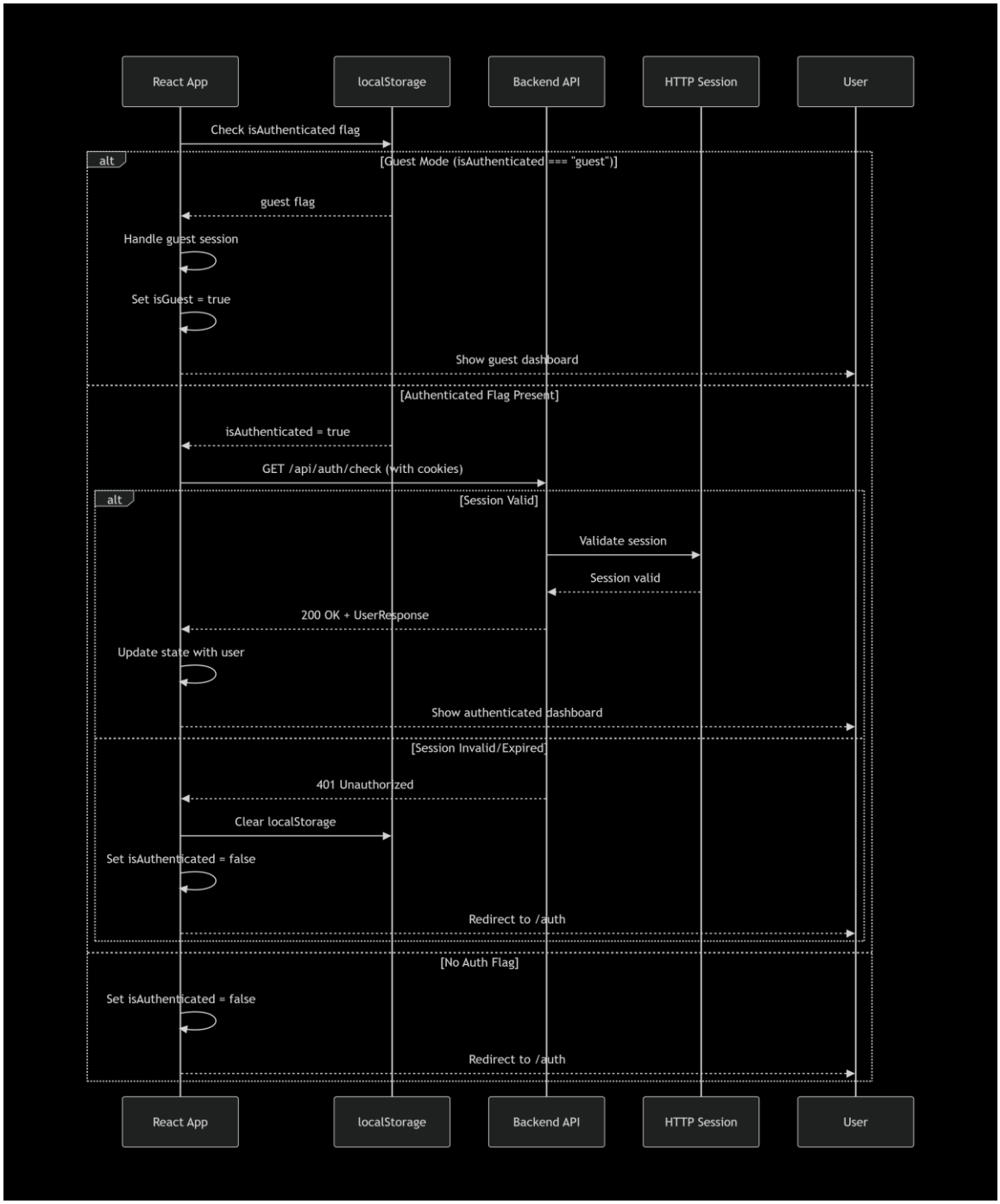
5.3. Activity Diagram



5.4. Class Diagram



5.5. Sequence Diagram



6. Appendices

Appendix A: API Endpoint Reference

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login and create session
POST	/api/auth/logout	Authenticated	Logout and invalidate session
GET	/api/auth/check	Public	Check current authentication
GET	/api/auth/test	Public	Health check
GET	/api/auth/health	Public	Health check
GET	/api/outfits	Public	Get all outfits
GET	/api/outfits/popular	Public	Get popular outfits
GET	/api/outfits/{id}	Public	Get outfit by ID

Method	Endpoint	Access	Description
GET	/api/outfits/category/{category}	Public	Get outfits by category
GET	/api/outfits/{id}/availability	Public	Check availability
GET	/api/user/me	Authenticated	Get current user profile
GET	/api/admin/**	ADMIN	Admin endpoints

Appendix B: Frontend State Management

LocalStorage Keys:

- isAuthenticated: Stores "true" for authenticated users, "guest" for guest users
- userRole: Stores role (USER, GUEST, ADMIN)
- userEmail: Stores authenticated user email
- userData: JSON string of complete user object

Application State (App.js):

- isAuthenticated: Boolean indicating if user is logged in
- isGuest: Boolean indicating guest mode
- user: Object containing current user details
- isLoading: Boolean for initial auth check

Appendix C: Database Schema (Future Implementation)

sql

```
-- Users table

CREATE TABLE users (
```

```
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(50) DEFAULT 'USER',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Outfits table
CREATE TABLE outfits (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(200) NOT NULL,
    category VARCHAR(50),
    description TEXT,
    price_per_day DECIMAL(10,2) NOT NULL,
    image_url VARCHAR(500),
    stock_quantity INT DEFAULT 1,
    designer VARCHAR(100),
    is_popular BOOLEAN DEFAULT FALSE,
    is_available BOOLEAN DEFAULT TRUE,
    rating DOUBLE DEFAULT 4.0,
    review_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Outfit sizes (one-to-many)
CREATE TABLE outfit_sizes (
    outfit_id BIGINT NOT NULL,
    size VARCHAR(20) NOT NULL,
    FOREIGN KEY (outfit_id) REFERENCES outfits(id) ON DELETE CASCADE
);

-- Bookings table (future)
CREATE TABLE bookings (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT NOT NULL,
    outfit_id BIGINT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    total_price DECIMAL(10,2),
    status VARCHAR(50) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (outfit_id) REFERENCES outfits(id)
);
```

Appendix D: Development Environment Setup

Backend Setup:

bash

```
# Clone repository
```

```
git clone [repository-url]

# Navigate to backend
cd eventwear-backend

# Configure database in application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/eventwear
spring.datasource.username=root
spring.datasource.password=password

# Run Spring Boot application
./mvnw spring-boot:run
```

Frontend Setup:

bash

```
# Navigate to frontend
cd eventwear-frontend

# Install dependencies
npm install

# Start development server
npm run dev
```

Configuration:

- Backend: http://localhost:8080
- Frontend: http://localhost:5173
- Database: MySQL on port 3306

Appendix E: Future Enhancements

1. **Booking Management**
    - Complete booking CRUD operations
    - Calendar view for availability
    - Booking confirmation emails
2. **Payment Integration**
    - Stripe/PayPal integration
    - Security deposits
    - Refund processing
3. **Advanced Features**
    - Outfit recommendations
    - User reviews and ratings
    - Wishlist/Save for later
    - Social media sharing
4. **Admin Dashboard**
    - Comprehensive admin interface
    - Analytics and reporting
    - Inventory management
    - User management
5. **Mobile Application**
    - React Native mobile app
    - Push notifications
    - Mobile-optimized booking